

Práctica 1. Microservicios. API REST en contenedor

Versión 1.7 - 12/09/2026

Introducción

En esta práctica implementaremos un servicio de almacenamiento de documentos de texto compartido. Este servicio se divide realmente en dos, el servicio de usuarios y el de documentos. Ambos se desplegarán en contenedores Docker como microservicios para poder realizar consultas y actualizaciones desde una API REST desarrollada en Python.

Objetivos docentes

Tras la realización de esta práctica, el estudiante debe ser capaz de:

- Desarrollar microservicios en Python que implementen una API REST.
- Desplegar microservicios en contenedores usando docker-compose.

Tecnologías a utilizar

Para esta práctica, se utilizará:

- Quart como framework ASGI (Asynchronous Server Gateway Interface). Quart es un entorno en Python para el desarrollo de aplicaciones web y micro-servicios.
- Docker como sistema de virtualización basado en contenedores para distribuir los servicios. La tecnología de contenedores permite aislar el entorno de ejecución de una aplicación del resto del sistema.

En esta práctica, todas las tecnologías se desplegarán en contenedores. Además, se implementará un único fichero *docker-compose.yml* para levantar y detener todos los contenedores y servicios necesarios.

Introducción a los conceptos de la práctica

1. Microservicios

Los microservicios son una alternativa opuesta a las aplicaciones monolíticas. Mientras que en una aplicación monolítica (todo en uno) toda la funcionalidad se encuentra en un único servidor, en una arquitectura de microservicios cada funcionalidad se desarrolla y despliega de forma independiente, desacoplada del resto.

Esto permite:

- Facilitar y acelerar cambios en la funcionalidad.
- Escalar sólo las piezas (funcionalidad) necesarias.
- Compartir recursos (funcionalidad) entre distintas aplicaciones.

La manera más extendida de implementar microservicios es mediante una API REST.

2. API REST

Una API REST es una interfaz que permite la comunicación entre aplicaciones a través del protocolo HTTP. Esta se basa en definir recursos, cada uno con una URL asociada. La gestión de los recursos se realiza mediante peticiones HTTP indicando la acción a realizar:

- GET: recuperación.
- PUT: creación o reemplazo completo.
- PATCH: actualización parcial.
- DELETE: eliminación.
- POST: creación o ejecución de acciones/procesos asociados.

NOTA: GET, PUT, PATCH y DELETE deben ser idempotentes, es decir, ejecutarlos varias veces consecutivas con los mismos parámetros debe producir el mismo efecto que ejecutarlo una única vez. POST no lo es.

Habitualmente, la información se intercambia en formato JSON, lo que facilita enormemente la interoperabilidad entre distintos sistemas y tecnologías.

3. Microservicios en Quart

Tanto Flask como Quart son entornos en Python para el desarrollo de aplicaciones web y microservicios. Quart es una evolución de Flask que optimiza su ejecución.

Los servicios web en Quart hacen uso de funciones asíncronas (*async def*), las cuales permiten seguir ejecutando otras tareas mientras se espera (*await*) a que termine la ejecución de otra función, típicamente debido a funciones de entrada/salida. Esto se hace en un solo hilo o proceso, evitando gran parte de los problemas de concurrencia típicos. Las funciones asíncronas son funciones capaces de esperar sin necesidad de intervención por parte del sistema operativo.

Para ejecutar los servicios, Quart usa las anotaciones *@app.route*, *@app.get*, *@app.put*, etc., que identifican qué función será necesario llamar dependiendo del método HTTP de la petición. Para más información, consulta la documentación oficial de Quart:

<https://quart.palletsprojects.com/en/latest/>

4. Entorno virtual en Python

Es conveniente trabajar con un entorno virtual en Python para cada práctica, lo que permite instalar dependencias distintas (y trabajar más fácilmente en los laboratorios docentes). Vamos a usar un entorno generado directamente con Python. Para ello, ejecuta en un terminal:

```
$> mkdir -p venv/silp1
$> python3 -m venv venv/silp1
$> source ./venv/silp1/bin/activate
$> pip install quart requests
```

El entorno virtual se desactiva (cuando se haya terminado de trabajar con él) con el comando `deactivate`. Con esto tendremos los paquetes necesarios para esta práctica. **Nota:** no se debe entregar el contenido de dicha carpeta `venv/silp1` al entregar la práctica. Para la entrega de la práctica, es suficiente con indicar las dependencias en un archivo `requirements.txt`. Para generar este archivo, usa (con el entorno virtual activado y una vez instaladas todas las dependencias):

```
$> pip freeze > requirements.txt
```

Luego, para instalar las dependencias en otro momento, se puede hacer con el comando:

```
$> pip install -r requirements.txt
```

5. Contenedores

La tecnología de contenedores permite aislar el entorno de ejecución de una aplicación del resto del sistema de forma que, por ejemplo, se pueda desarrollar una aplicación que requiera de un entorno (paquetes) en un sistema operativo y se quiera ejecutar en otro. También permite disponer de software propio en ese entorno de ejecución; por ejemplo, se pueden tener versiones anteriores a las instaladas en el sistema o software nuevo que no existan en dicho sistema.

Es parecido a lo que hace con el entorno de Python usado anteriormente, pero a una escala mucho mayor, sin llegar a la independencia que da una máquina virtual.

5.1. Modo *rootless*

El gestor (demonio) de contenedores se puede ejecutar de distintas formas: en modo *rootful* (con permisos de superusuario) o *rootless* (como un usuario normal). También es posible ejecutar un gestor de contenedores anidado (docker in docker) o virtualizado (como Docker Desktop).

En el modo *rootful*, los contenedores se arrancan con prácticamente todos los privilegios de root en un entorno aislado. Esto presenta vulnerabilidades que exigen un control exhaustivo de los contenedores desplegados.

En modo rootless, el gestor se arranca con los privilegios del usuario que lo arranca y no presenta estos peligros, aunque también presenta una serie de limitaciones, si bien se está haciendo un esfuerzo importante (incluso a nivel del kernel de linux) para resolverlas.

Docker Desktop es una forma de ejecutar el gestor docker como si fuera rootful, pero dentro de una máquina virtual, por lo que se pierde algo de eficiencia, aunque ganamos en seguridad y compatibilidad. No es un sistema apropiado en producción, pero facilita el desarrollo, ya que además posee una interfaz gráfica y resulta conveniente para el uso en ordenadores personales.

5.2. Ejecución en los laboratorios

El modo rootful de docker está desactivado por cuestiones de seguridad y no lo usaremos en los laboratorios. Para iniciar Docker en los laboratorios se debe ejecutar en un terminal:

```
# Establecer los permisos de ejecución para el usuario
$> sudo /usr/local/bin/si2setuid.sh 3000000:100000 $USER
# Arrancar el demonio dockerd
$> dockerd-rootless-setuptool.sh install --skip-iptables
$> export DOCKER_HOST=unix:///run/user/$(id -u)/docker.sock
# Mostrar la info del demonio docker
$> docker info
```

5.3. Docker Compose

Para arrancar automáticamente diferentes servicios relacionados en Docker se puede crear un archivo `docker-compose.yml`. Cada servicio, asociado a un contenedor, será independiente y podrá escuchar un puerto distinto. Además, se puede configurar dónde se guardarán realmente los datos, configurando un volumen distinto para cada servicio.

Tareas a realizar

En esta práctica implementaremos un servicio de almacenamiento de documentos de texto compartido. Este servicio se divide realmente en dos, el servicio de usuarios y el del directorio de documentos.

Cada usuario tendrá un directorio de documentos y podrá añadir, listar, descargar, o borrar documentos de su directorio. En el caso de que se añada un documento y este ya exista, se reemplazará su contenido.

Además, cada documento podrá marcarse como público o privado.

- Los documentos públicos podrán ser descargados por otros usuarios si conocen el ID del usuario (UID, véase la siguiente sección) del propietario y el nombre del documento.
- Los documentos privados no podrán ser descargados por terceros.

- El propietario siempre podrá listar y gestionar sus documentos (crear, actualizar, borrar y cambiar su visibilidad).

El resto de los usuarios podrán descargar el contenido de los documentos públicos de otros usuarios, siempre que conozcan el UID y el nombre del documento, pero no podrán listar, o modificar el directorio de otros usuarios.

1. El Sistema de UIDs

Para que el sistema sea seguro, ha de permitir crear usuarios con un nombre y una contraseña (*create_user*). La contraseña deberá almacenarse mediante un hash seguro. Para esta práctica basta con utilizar SHA-256. Al crear un usuario se le asignará un identificador único (UID). Este UID se puede crear con la función `uuid.uuid4()` que genera un ID aleatorio suficientemente grande como para que estemos casi seguros de que es único. Si todo es correcto, se devolverá dicho UID junto con el token de acceso que, para simplificar el proceso, será un hash (SHA-1) derivado del UID, por lo que el servicio de directorio podrá verificar fácilmente si el token es válido. De esta forma, *secret_uuid* será conocido por ambos servicios (se generará una única vez), pero no por el cliente. En el caso de la librería UUID de Python, *secret_uuid* es a su vez un UID, y usaremos la función `uuid5` para usar SHA-1 como hash. Ejemplo:

```
secret_uuid = uuid.uuid4() # Este se habrá generado una única vez y guardado en el servidor
uid_del_usuario_como_string = str(uuid.uuid4()) # Este se habrá generado al hacer create_user
token = uuid.uuid5(secret_uuid, uid_del_usuario_como_string)
```

Aviso: En un caso real, para mayor seguridad se deben usar otros mecanismos mejores para crear los tokens, que incluyan un tiempo de expiración y no requieran que todos los servicios conozcan la clave *secret*, por ejemplo, usando *tokens JWT* con expiración y firma con un sistema de clave pública/privada. Esto es solo un ejercicio didáctico.

Un usuario puede entrar al sistema de forma similar (*login*), indicando su nombre y contraseña, y en caso de éxito, el sistema le devolverá su token de acceso y el UID. Tanto el UID como el token de acceso serán necesarios para poder acceder al directorio de un usuario. Además, el servicio de usuarios debe permitir cambiar la contraseña de un usuario autenticado (*modify_user*).

2. API REST

En esta práctica se implementará un prototipo de los servicios *user* (usuarios) y *file* (directorios), a través de los ficheros *user.py* y *file.py*, respectivamente. No es necesario gestionar el directorio como un recurso distinto a los archivos, ya que solo tendremos un directorio por usuario, por lo que no tiene sentido crearlos o destruirlos, aunque sí es posible listar su contenido.

En el caso de *user* no hay que añadir nada a la ruta, y se usará simplemente la ruta `/user`. La información con el nombre, o la contraseña se añadirá al cuerpo de la petición, usando los campos *name* y *password*, respectivamente. Tal y como se ha descrito anteriormente, el servicio de usuarios devolverá el token y el UID del usuario en el cuerpo de la respuesta cuando se haga

Sistemas Informáticos. Curso 2026/2027

Escuela Politécnica Superior. Universidad Autónoma de Madrid

create_user o *login*, usando los campos *token* y *uid*. En ambos casos el cuerpo de la petición/respuesta estará en formato JSON.

Respecto a la implementación del servicio *file*, el directorio de un usuario estará accesible en la ruta */file/<UID>*. Por tanto las rutas de los recursos de tipo archivo serán: */file/<UID>/nombre_archivo*. Debéis implementar el mecanismo necesario para gestionar los documentos públicos y privados de un directorio.

En el caso del token de seguridad para acceder al servicio de archivos, este se pasará en el campo *Authorization*, en la cabecera de la petición, indicando que es de tipo *Bearer*. Por ejemplo:

`Authorization: Bearer 0b43d6ba-b431-4fee-b649-1d14a871ec00.93096740305b3341c`

Se aconseja desarrollar este apartado siguiendo los siguientes pasos:

1. Creación y pruebas del microservicio de directorios, sin comprobación de los tokens. Para las pruebas se creará un cliente en Python que llame a la API del servicio. No es necesario que el cliente envíe o reciba archivos reales, ya que al ser documentos de texto se podrían enviar cadenas de texto, y mostrar dicho texto al obtenerlos del servidor. Esto además puede facilitar las pruebas. En cuanto a la persistencia del servicio, aunque es de libre elección, se recomienda usar una estructura sencilla de directorios y archivos de texto.
2. Creación del microservicio de usuarios, añadiendo la comprobación de los tokens al servicio de directorio y el cliente de pruebas. Para esta práctica se recomienda usar la librería *uuid* para generar los UUIDs de forma aleatoria.

Nota: Cada microservicio será un programa distinto, que escuchará en un puerto diferente, mientras que para las pruebas se puede usar un mismo programa cliente que llame a ambos servicios. Por otro lado, deben devolverse los códigos de error apropiados (HTTP) para cada petición.

Resumen ilustrativo de endpoints:

Ruta	Método	Parámetros	Descripción
<i>/user</i>	PUT	name, password	Crea usuario
<i>/user</i>	POST	name, password	Inicia sesión
<i>/user</i>	PATCH	password	Modifica la contraseña de un usuario autenticado
<i>/file/<uid></i>	GET	UID	Lista documentos de usuario
<i>/file/<uid>/<filename></i>	PUT	UID, filename	Crea o actualiza documento de usuario
<i>/file/<uid>/<filename></i>	GET	UID, filename	Recupera documento de usuario
<i>/file/<uid>/<filename></i>	DELETE	UID, filename	Elimina documento de usuario
<i>/file/<uid>/<filename></i>	PATCH	UID, filename, public	Modifica la visibilidad (pública) de un documento de usuario

Algunos datos deben ir en el cuerpo de la petición (JSON, headers). El parámetro *<uid>* identifica al propietario de los documentos. El servicio deberá utilizar dicho identificador para localizar los

documentos almacenados. Cuando una operación requiera autenticación, el servidor verificará que el token recibido corresponde al UID indicado en la ruta. Los documentos privados únicamente podrán ser recuperados por su propietario. Los documentos públicos podrán recuperarse sin autenticación mediante la ruta `/file/<uid>/<filename>`.

3. Cliente de prueba

En las primeras pruebas, puedes usar el navegador para probar las rutas de la API que usen GET, o el comando `curl` para poder enviar datos en formato JSON, o usar distintos métodos HTTP. Por ejemplo, para crear un usuario:

```
curl -X PUT http://127.0.0.1:5050/user -H 'Content-Type: application/json' -d
'{"name":"alice","password":"my_password"}'
```

El comando anterior realiza una petición PUT al puerto 5050, en la ruta `/user`, y envía un objeto JSON con un campo con el nombre (*name*) del usuario *alice* y la contraseña (*password*) con el valor *my_password*. En la petición HTTP, podemos añadir cabeceras con la opción `-H`. En este caso se ha especificado que el tipo del contenido es `application/json`.

Lo que debéis entregar para probar que todo funciona es un cliente en Python (`cliente.py`) que interactúe con ambos servicios. Para esta práctica se creará un cliente sencillo, no interactivo, utilizando el paquete *requests*, que simplemente indique el nombre de la prueba y si el resultado es el esperado o no. Se deberán probar todas las rutas implementadas, intentando que con la ejecución del cliente se ejecute todo el código de los servicios, incluidos los casos de error implementados. Puedes obtener más información en: <https://pypi.org/project/requests/>.

Planificación

Se recomienda la siguiente planificación para desarrollar la práctica:

Semana 1	Codificación de la API: <code>user.py</code>
Semana 2	Codificación de la API: <code>file.py</code>
Semana 3	Cliente (<code>cliente.py</code>), memoria, dudas, etc.

Entrega

La fecha concreta y normas de entrega de las prácticas se encuentran en Moodle. Como mínimo se deben entregar los siguientes archivos:

- Se entregará un único fichero ZIP estructurado (`P1.zip`) que contenga el proyecto completo con los ficheros necesarios para su ejecución. Esto incluye los ficheros fuente en Python, scripts, dependencias, fichero YAML, etc. Se debe incluir un fichero `README.txt`

en la raíz con información e instrucciones básicas de despliegue y ejecución local. No se deben entregar entornos virtuales, cachés, bytecodes ni metadatos del sistema operativo.

- Memoria de la práctica con una descripción y capturas de la resolución de cada apartado.

Rúbrica de corrección

Se seguirá la siguiente rúbrica para la calificación de la práctica, donde se tendrá en cuenta la calidad, completitud y tolerancia a fallos de lo implementado:

Concepto	Calificación Máxima
Servicio users.py	3
Servicio file.py	3
Cliente de pruebas cliente.py	2
Gestión de Docker (Dockerfile de los servicios y docker-compose)	1
Memoria (debe contener una primera página con los autores, grupo y equipo)	1
TOTAL	10

Se aplicará la penalización de 1 punto por día natural (o fracción) de retraso en la entrega. De la misma forma, se sancionará, según normativa UAM, la copia o uso indiscriminado de la IA.